Curriculum Intelligence Engine — Week 6: Asset Engineering (reusable NLP asset)

Group 9 · Roll Nos: J048, J049, J050, J052, J053 · AI University NLP Course Project 2026 · Week 6

CODE

```
Repository : https://github.com/jainamsethia/Curriculum-Intelligence-Engine
Tag        : week-6-redo-fix
Commit     : ee7c1cdf31ee65b973018afccbf33f69968e173f
Install    : pip install -e .[dev]
Serve      : uvicorn curriculum_engine.api:app --port 8000       (OpenAPI docs at /docs)
Reproduce  : python -m curriculum_engine.run all                 (data → week2 … week6; NMIMS_ZIP or
data/processed/courses.jsonl)
Tests      : python -m pytest -q                                  (unit tests + API tests on a synthetic fixture)
New        : curriculum_engine/api.py, api_engine.py, schemas.py, prereq.py (checker v2), experiments/week6.py, docs/API.md
Folders    :
.gitignore
README.md
curriculum_engine/   the NLP engine (parser, data, overlap, redundancy, prerequisites, diff, Bloom, evaluation)
docs/                documentation
labels/              LLM-annotated reference labels (ids + labels per pass + consensus)
legacy/              old Week 3-4 scripts and reports (history only)
legacy_mit/          old MIT OCW scripts (not used)
pyproject.toml
reports/             report templates (numbers are placeholders)
requirements.txt
results/             metrics JSON + figures per week
submissions/         per-week submission folders
tests/               unit tests (pytest)
tools/               report builder, report checker, submission packer
```

Feedback addressed

Comment	What we changed	Evidence
Week 4: "Where is the code? … overlapping content, missing pre-requisites, detect redundancies"	The three core capabilities are now endpoints of an installable package and API with structured input, output and confidence.	`curriculum_engine/api.py`, `docs/API.md`
Our Week 5 error analysis: prerequisite flags were mostly wrong (cases F05–F07)	Prerequisite checker v2: phrase cleaning, non-subject filter, alias / umbrella names, own-content filter; re-evaluated on the same split and labels.	`results/week6/prerequisites_v2.json`
Week 2: "an asset that will be of value to NMIMS"	Every answer runs on our NMIMS semester syllabus booklets and can also take a new syllabus as raw text.	`docs/examples/`

Progress vs previous week

Item	Week 5	Week 6
Reusable asset	scripts and a Python engine class	installable package `curriculum_engine` version 0.6.0, FastAPI service with 7 endpoints, Pydantic schemas, OpenAPI
Missing-prerequisite flags on test courses	33	14
Prerequisite flag precision, fresh validation sample	0.200	0.450

1. Prerequisite checker v2 (before / after, honest)

The Week 5 failure cases showed why most missing-prerequisite flags were wrong: umbrella names ("Engineering Mathematics" when Calculus and Linear Algebra are taught), fragments created by splitting free text ("III", "Even", "Management"), and phrases that are not subjects ("subjects studied till semester V", "geography", topics of the course itself). Checker v2 adds four steps before a flag is raised:

- **Phrase cleaning**: parentheses and course codes are dropped; "and" / "or" only split a phrase when both sides have two or more words; a bare roman numeral is re-attached to the preceding course name.
- **Non-subject filter**: phrases such as "all courses studied till …", generic skills and school subjects are skipped.
- **Alias / umbrella names**: the prerequisite counts as met if every content word of the phrase, or one member subject of a broad subject (mathematics → calculus, algebra, differential equations …), appears in the name of one earlier course.
- **Own-content filter**: a phrase that the course's own units teach is course content, not a prerequisite (threshold 0.600, chosen on validation).

Measure	v1 (Week 3)	v2 (Week 6)
Removal recall, test cases	0.880	0.711
False alarms, intact programme	0.036	0.000
False alarms after renaming the course	0.120	0.000
Precision on the same 20 labelled test flags	0.100	0.222 on the 9 flags v2 still raises
Precision of all v2 test flags (fresh labels where the phrase changed)	–	0.500 [0.176, 0.875] (n = 14)
Precision, fresh validation sample	0.200 [0.050, 0.500]	0.450 [0.208, 0.737]

- On the same labels, v2 drops 11 flags that were judged wrong and 0 that were judged correct, and its false alarms in the label-free tests fall to zero.
- The cost is recall: v2 misses more removed prerequisites, because another earlier course with the same subject words still counts (removing "Engineering Mathematics - I" while "Engineering Mathematics - II" is taught is arguably fine, which the removal test cannot see). Removal recall is now below the target of 0.800, and precision is still below the target of 0.600.
- Caution: the rules were designed from test-split failure cases, so the after-numbers on test labels are optimistic. The fresh validation sample is less biased; its intervals overlap, so the gain is indicative, not proven. A first attempt that pooled words across all earlier course names (removal recall 0.627) was replaced; its results stay in the results file. The 45 fresh items were labelled with the Week 3 flag protocol: two Claude passes with different wording agreed on 45 of them, so pass 3 was needed for 0 items (same model family: this agreement is not independent).

2. The reusable asset

- **Package**: `curriculum_engine` (pyproject, `pip install -e .`), entry point `python -m curriculum_engine.run`; the API engine `ApiEngine` can be used directly from Python.
- **API** (FastAPI, Pydantic v2 schemas, OpenAPI at `/docs`, exported to `docs/openapi.json`): `POST /compare-curricula` (two courses, or two programmes / two years of one programme), `POST /map-outcomes`, `POST /find-overlaps`, `POST /check-prerequisites`, `POST /detect-redundancy`, `POST /emerging-topics`, `GET /health`.
- **Structured input**: a course is an id, a course name or raw syllabus text; a programme is a code plus an optional academic year. Invalid requests get `422`, unknown courses or programmes `404`.
- **Confidence scores** are defined from the evaluation, not invented: overlap confidence = P(any overlap) from the Week 4 logistic model (relative, not well calibrated); prerequisite and redundancy flags carry their measured precision (0.500 and 0.800); a Bloom level carries the precision of that level against faculty tags (Create 0.429, Understand 0.704, Apply 0.786).
- **Versioning**: every response has `meta` (engine version, encoder, number of course versions, split hash).
- **Examples and tests**: `docs/examples/*.json` are real responses regenerated by `run week6`; status codes: compare (courses) 200, compare (programmes) 200, map-outcomes 200, find-overlaps 200, check-prerequisites 200, detect-redundancy 200, emerging-topics 200, health 200. API tests run on a synthetic programme, so they need no NMIMS data.
- **Docker**: not provided — Docker is not installed on the development machine.

Example: a real request and response of `POST /compare-curricula` (shortened; full version in `docs/examples/compare_courses.json`):


```
{
 "request": {
  "course_a": {
   "id": "nm48af830b"
  },
  "course_b": {
   "id": "nm9d8c41ec"
  }
 },
 "response": {
  "course_a": {
   "id": "nm48af830b",
```

```
   "course_name": "Department Elective IV - Supply Chain Analytics",
   "code": "BTME08016",
   "academic_year": "2021-22",
   "semester": "VIII",
   "programmes": [
    "MECHANICAL"
   ]
  },
  "course_b": {
   "id": "nm9d8c41ec",
   "course_name": "Open Elective I - Logistics and Supply Chain Management",
   "code": "MBOEMEL001",
   "academic_year": "2021-22",
   "semester": "VII",
   "programmes": [
    "MECHANICAL"
   ]
  },
  "overlap": {
   "percent": "91%",
   "semantic_similarity": 0.791,
   "lexical_similarity": 0.698,
   "confidence": 0.833,
   "redundancy_warning": true
  },
  "common_topics": [
   {
    "topic_a": "Designing the Supply Chain Network",
    "topic_b": "Designing the Supply Chain Network",
    "similarity": 1.0,
    "topic_label": "marketing, supply chain, customer"
   },
   {
    "topic_a": "Warehouse and Transportation Management",
    "topic_b": "Warehouse and Transportation Management",
    "similarity": 0.961,
    "topic_label": "waste, industrial, quality"
   },
   {
    "topic_a": "Building a Strategic Frame Work to Analyze Supply Chain",
    "topic_b": "Building a Strategic Framework to Analyze Supply Chain",
    "similarity": 0.943,
    "topic_label": "marketing, supply chain, customer"
   }
  ],
  "unique_to_a": [
   "Supply chain Analytics-I"
  ],
  "prerequisites_a": {
   "checked": true,
   "reason": null,
   "flags": []
  },
  "meta": {
   "engine_version": "0.6.0",
   "encoder": "sentence-transformers/all-MiniLM-L6-v2",
   "split_hash": "4763325fd5"
  }
 }
}
```


3. Limitations

- Data: NMIMS semester syllabus booklets; 531 of 2,413 non-exam PDFs are parsed, the rest are scanned and need OCR (future work), so the API can report a course as removed or a prerequisite as missing when the evidence sits in an unread booklet.
- Labels and confidence values come from LLM-annotated reference labels. No human annotators were used; results are indicative. For 94 of 200 pairs the second labeller was a Claude prompt rather than Phi-3, so agreement on those items is not independent. Detector flags (Week 3 onwards) were labelled by two Claude passes with different wording plus a pass 3, with no Phi-3, so agreement on flags is not independent.
- Redundancy pairs include templated units (project or case-study units identical across courses).
- The parsed dataset is in the Teams code zip only, never in the public repository.

AI assistance

Code and report text were written with AI assistance (Claude). Reference labels are LLM-annotated (Claude, Phi-3). Every number is generated from `results/*.json` by `tools/build_report.py` and verified by `tools/check_reports.py`.